Hybrid Rule and Isolation Forest Anomaly Detection for Microservice Logs on MongoDB with Evidence-Grounded LLM Incident Investigation

G Sivakumar
Reg. No. 23BCE1834
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India

Afzal Ahamed A
Reg. No. 23BCE1868
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India

***Abstract*—Microservice platforms emit large volumes of logs, and operators need to notice failures quickly and understand where they started. This paper presents a log monitoring system built on MongoDB that summarizes each service's logs into overlapping five-minute windows of 19 features and scores every window with two complementary detectors: a severity-weighted rule engine and an unsupervised Isolation Forest. The two confidences are fused with fixed weights (0.4 and 0.6) into a single decision. When an incident occurs, an investigator correlates anomalies across services, reconstructs a minute-level timeline from the raw logs, ranks root-cause hypotheses with a service dependency map, and asks a locally hosted large language model to write a report that must cite numbered evidence; citations are verified and state-changing advice is flagged for human approval. On five labelled runs (5,272 windows), the hybrid detector reaches an F1-score of 0.870 and accuracy of 0.983 without any labels, ahead of Local Outlier Factor (0.371) and One-Class SVM (0.608). A supervised Random Forest scores higher (F1 0.931) when it is trained on 4,218 labelled windows of every incident type, but it detects none of the error bursts or traffic drops once that incident type is absent from its training data, whereas our Isolation Forest still recalls 84-85% of them. The investigator identified the correct origin of a three-service cascading failure in five of five runs.**

***Index Terms*—anomaly detection, log analysis, isolation forest, random forest, MongoDB, microservices, root cause analysis, large language models**

I. Introduction

Modern applications are split into many small services that call one another over the network. A fault in one service rarely stays inside it: a slow database behind a user service makes the authentication service time out, which in turn makes payments fail. Each service writes its own log stream, so an operator who looks at a failing payment service sees only symptoms. Two questions therefore matter during an incident: whether something is wrong right now, and which component started the problem.

Fixed threshold alerts answer the first question cheaply and are easy to interpret, but they only catch conditions that someone anticipated. Machine-learning detectors can flag unusual behaviour that no rule describes, yet supervised models need labelled examples of failures, which are scarce and quickly outdated, and unsupervised models tend to raise false alarms. Neither kind of detector explains where a multi-service failure began.

This work combines both detectors and adds an evidence-driven investigation step. Its contributions are:

- A MongoDB data design for log monitoring that keeps raw logs, window features, detection results, rules, versioned models (in GridFS) and incident reports in one store, using compound indexes, aggregation pipelines and time-to-live expiry.
- A hybrid detector that fuses a severity-weighted rule confidence with a normalized Isolation Forest score, needs no labelled data, and explains every alert with the rules, values and thresholds involved.
- An incident investigator that correlates anomalies across services, finds each service's onset from raw logs, scores root-cause hypotheses with a dependency map, and grounds a local LLM's report in verified evidence without allowing any automatic action.
- An evaluation against Random Forest, Local Outlier Factor and One-Class SVM, including a test on incident types that were never seen during training.

II. Related Work

Early log anomaly detectors treated each session as a vector of event counts. Xu et al. applied principal component analysis to console logs [5], and Lou et al. mined linear invariants between event counts [6]. He et al. compared these and several supervised methods on public datasets and released the loglizer toolkit [7]; a later survey summarizes the field [14]. Deep models then learned event sequences: DeepLog predicts the next log key with an LSTM [8], LogAnomaly adds semantic templates [9], and LogBERT trains a transformer with self-supervised objectives [10]. Le and Zhang showed that the reported accuracy of deep methods drops under realistic data splits [11]. These approaches focus on the order of log messages; they do not use response time or traffic volume, and they output a binary label without explaining it.

Isolation Forest isolates outliers with random partitions and needs no labels [1]. Random Forest is a strong supervised ensemble [2], while Local Outlier Factor [3] and One-Class SVM [4] are common unsupervised baselines. Recent studies use large language models to suggest root causes and mitigations for cloud incidents [12], [13]. Language models

can, however, state facts that are not supported by their input [17], which is risky during an outage. Our investigator therefore computes the root cause from data first and only lets the model explain it, with every statement checked against the evidence.

III. System Architecture

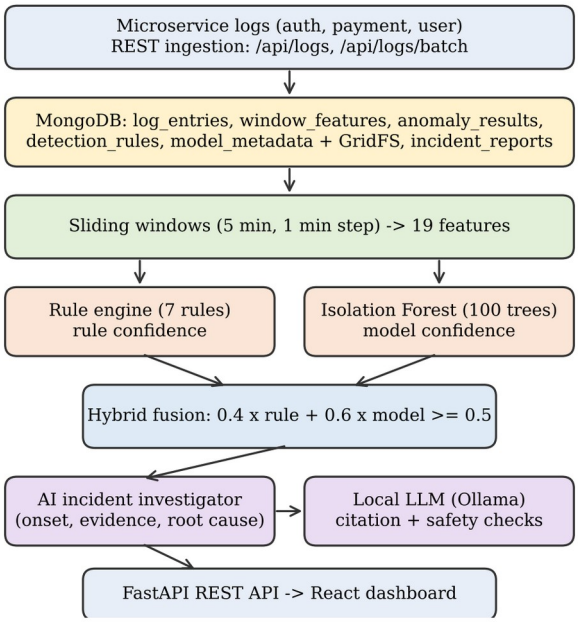


Fig. 1. Data flow of the system, from log ingestion to the dashboard.

Fig. 1 shows the pipeline. Services send logs to a FastAPI backend through single or batch ingestion endpoints. Each log carries a timestamp, service name, level, message, endpoint, HTTP method, status code, response time, payload sizes and a free-form metadata field. Logs are stored in MongoDB, summarized into windows, scored by the two detectors, and fused into a decision that is stored together with its explanation. A React dashboard reads the results through REST endpoints, and the investigator is invoked from the dashboard for a selected anomaly.

A. MongoDB Data Design

The database holds seven collections. **log_entries** stores raw logs; its flexible document schema lets each service attach different metadata without migrations. **window_features** stores one document per service and window with the 19-value feature vector. **anomaly_results** stores every decision with the rule, model and fused confidences and the details of each violated rule. **detection_rules** keeps the rule definitions as data, so thresholds can change without code changes. **model_metadata** records model versions and parameters, while the serialized model itself is stored in **GridFS** because it exceeds the size intended for ordinary documents. **incident_reports** keeps investigation results.

Compound indexes on (service_name, timestamp) and (window_start, service_name) serve the dashboard's most frequent query, the latest records of one service. Time-to-live indexes remove logs after 30 days, windows after 90 days and results after 180 days, so storage stays bounded without a cleanup job. Aggregation pipelines compute averages, the 95th percentile latency with $percentile, error counts and per-minute timelines inside the database rather than in application code.

IV. Methodology

A. Windowing and Features

Logs of each service are grouped into windows of five minutes that start every minute, so consecutive windows overlap and a problem is seen within about a minute. A window is scored only if it contains at least five requests. Table I lists the features computed per window.

TABLE I
Window features

Group	Features
Volume	request count, error count approximation
Errors	error rate (ERROR logs / requests)
Latency	mean, 95th and 99th percentile, maximum, time per request
Payload	mean request size, mean response size
HTTP methods	GET, POST, PUT, DELETE, PATCH counts
Status codes	2xx, 3xx, 4xx, 5xx counts

B. Rule Engine and Rule Confidence

Seven rules encode operational limits (Table II). Threshold rules compare a feature with a fixed limit. Rate-change rules compare it with the mean of the previous three windows; for error rate this mean is floored at 5% so that a change from zero to one error is not reported as a large relative spike.

TABLE II
Detection rules

Rule	Condition	Severity
high_error_rate	error rate > 10%	high
very_high_error_rate	error rate > 25%	critical
high_latency_p95	P95 > 1000 ms	medium
high_latency_p99	P99 > 2000 ms	high
low_traffic	requests < 10	medium
sudden_traffic_drop	requests < 0.5 x recent	high
sudden_error_spike	error rate > 3 x recent	high

Each violated rule i receives a score s_i in [0, 1]. For threshold rules the score grows with the relative distance beyond the limit, $s_i = \min(1, |v - t| / t)$, where v is the observed value and t the threshold; rate-change rules receive $s_i = 0.8$. Each rule also has a severity weight w_i (critical 1.0, high 0.8, medium 0.6). The rule confidence is the weighted mean

$$R = \Sigma\, w_i\, s_i \,/\, \Sigma\, w_i \qquad (1)$$

and R = 0 when no rule fires. For example, a window with 81.8% errors and P95 of 4940 ms violates six rules; four threshold rules score 1.0 and two rate-change rules score 0.8, which gives R = 4.48 / 4.8 = 0.93.

C. Isolation Forest and Model Confidence

Isolation Forest builds random binary trees in which every node splits on a randomly chosen feature at a random value [1]. Points that differ from the bulk of the data are separated

after few splits, so a short average path length indicates an anomaly. We train 100 trees, each on a random 80% of the windows, after standardizing every feature to zero mean and unit variance so that millisecond-scale latency does not dominate rates between 0 and 1. The contamination parameter is set to 0.1, which places the decision boundary at the most isolated 10% of training windows. Training uses no labels and takes well under a second; the fitted model and scaler are versioned and stored in GridFS.

The library's decision score d is zero at the boundary and negative for anomalies. We map it to a confidence in [0, 1] with a steep logistic function

$$M = 1 / (1 + e^{20\,d}) \qquad (2)$$

so that the boundary corresponds to M = 0.5 and typical scores between -0.2 and 0.2 spread across the whole range. With a slope of one, almost all normal windows had confidences between 0.48 and 0.50 and were mislabelled as borderline. A window with 0.4 <= M < 0.5 is borderline; three consecutive borderline windows raise an alert, which catches slow degradations that never look extreme in one window.

D. Hybrid Fusion

The two confidences are combined as

$$F = 0.4\,R + 0.6\,M, \text{ anomaly if } F \geq 0.5 \qquad (3)$$

The model receives the larger weight because it generalizes to patterns no rule describes. With these weights, strong rule evidence (R = 1) needs only M ≥ 0.17 to alert, while the model alone needs M ≥ 0.84, which suppresses many of its false alarms. The fused decision is labelled hybrid_both when both detectors flag the window, rule_only or model_only otherwise, and borderline_persistence when the persistence rule fired. Until a model exists the system falls back to rules alone. Repeated alerts for the same service and the same violated rules within five minutes of window time are stored but not raised again.

E. Real-Time and Batch Detection

When logs arrive through the API, the backend builds the newest window of that service from the last ten minutes, scores it, and stores both the window and the decision; a decision takes 11 to 25 ms. A batch pipeline processes all stored logs in time order, trains the model if none is stored, and is exposed through a command-line tool and an API endpoint.

F. Evidence-Grounded Incident Investigation

The investigator starts from one anomaly and collects the anomalies of all services within 15 minutes before and after it. For each affected service it computes a baseline from the 60 windows preceding the incident and scans the raw logs minute by minute. A minute is abnormal if it has at least three errors and an error rate of at least max(30%, 5x baseline), or a mean latency of at least three times the baseline. The onset is the first minute of a run of at least two abnormal minutes, which prevents a single noisy minute from being taken as the start.

Findings are recorded as numbered evidence (E1, E2, ...): anomaly records, metrics compared with the baseline, the onset minute, and the most frequent error messages, each pointing to its MongoDB source. Every affected service c is then scored as a root-cause candidate:

$$S_c = 0.3\,C_c + 0.25\,f_c + 0.25\,D_c + 0.2\,m_c \qquad (4)$$

where C_c is the highest anomaly confidence of c, f_c is 1 if c degraded first, D_c is the fraction of other affected services that depend on c and degraded after it, and m_c is 1 if error messages of other services name c. A single affected service receives 0.2 more, and scores are capped at 0.99. Severity, read-only troubleshooting steps and a timeline are derived from the same evidence.

The computed conclusion, alternatives, timeline and evidence are sent to a local model (llama3.2 through Ollama, temperature 0.1) with instructions to cite evidence identifiers in every sentence and never to recommend state-changing actions. The response is checked: citations to unknown identifiers and sentences without citations are listed for the reader, and words such as restart, delete or roll back are flagged as requiring human approval. The investigator only reads data and the model has no tools, so nothing is executed automatically. If the model is unavailable, the data-driven part of the report is still produced. Running the model locally keeps logs inside the organization and avoids API cost.

V. Implementation

The backend uses Python, FastAPI, PyMongo and scikit-learn [16]; the dashboard uses React, Material UI and Recharts. The dashboard shows system totals, per-service latency and error metrics, trend charts, and tables of anomalies and logs (Fig. 2). The details view of an anomaly lists the fused, rule and model confidences and every violated rule with its value, threshold and severity (Fig. 3). A button starts the investigation (Fig. 6), and another simulates a cascading failure in which the user service's database slows down, the authentication service then times out calling it, and the payment service fails one minute later.

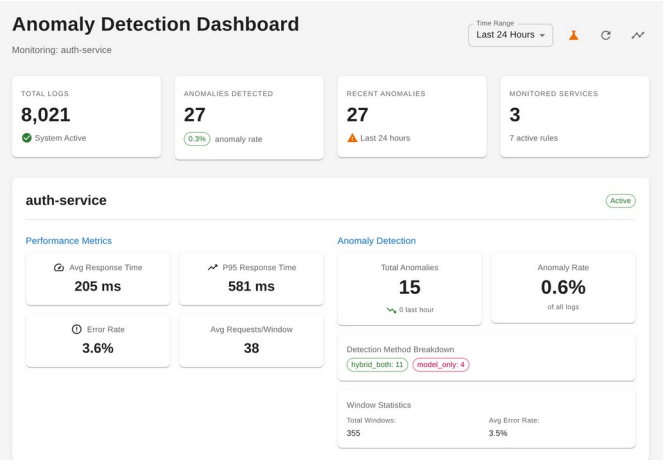


Fig. 2. Monitoring dashboard with system totals, per-service metrics and the breakdown of detection methods.

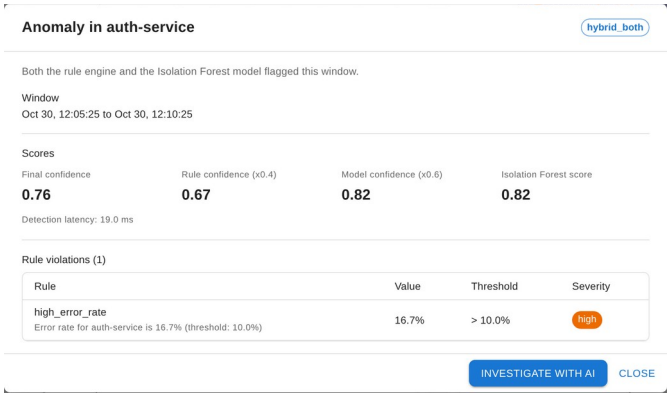


Fig. 3. Explanation of one alert: fused confidence 0.76 = 0.4 x 0.67 + 0.6 x 0.82, with the violated rule, observed value and threshold.

VI. Experimental Setup

Labelled anomalies are not available for real microservice traffic, and public log benchmarks such as HDFS record event sequences without response times. We therefore generated labelled traffic with a seeded generator. Each run covers six hours for three services at about eight requests per minute (about 8,000 logs, 1,054 windows) with log-normal latencies around 135 ms and 2% background errors. Four incidents are injected: a 15-minute latency spike in the payment service, a 30-minute gradual slowdown and a 10-minute error burst in the authentication service, and a 15-minute traffic drop in the user service. A window is labelled anomalous if it contains logs of an injected incident. Five runs with different seeds give 5,272 windows, of which 375 are anomalous.

The unsupervised methods (our rule engine, Isolation Forest and hybrid, Local Outlier Factor with 20 neighbours and One-Class SVM with nu = 0.1) are fitted on the unlabelled windows they monitor. Random Forest (100 trees) is supervised: for each run it is trained on the labelled windows of the other four runs (4,218 windows) and tested on the held-out run. Experiment A gives Random Forest examples of every incident type. Experiment B removes one incident type from its training data and measures recall on that type, which models a failure mode that has not occurred before. Precision, recall, F1-score, accuracy and false alarms are summed over the five runs. The scripts evaluation/model_comparison.py and evaluation/investigation_eval.py reproduce all numbers.

VII. Results and Discussion

A. Comparison with Random Forest and Unsupervised Baselines

TABLE III
Detection results over five runs (Experiment A); Alarms = false alarms

Model	Labels	Prec.	Rec.	F1	Acc.	Alarms
Rules (ours)	no	0.901	0.752	0.820	0.976	31
IF (ours)	no	0.638	0.901	0.747	0.957	192
Hybrid (ours)	**no**	**0.950**	**0.803**	**0.870**	**0.983**	**16**
Random Forest	yes	0.963	0.901	0.931	0.991	13
LOF	no	0.317	0.448	0.371	0.892	362
One-Class SVM	no	0.521	0.731	0.608	0.933	252

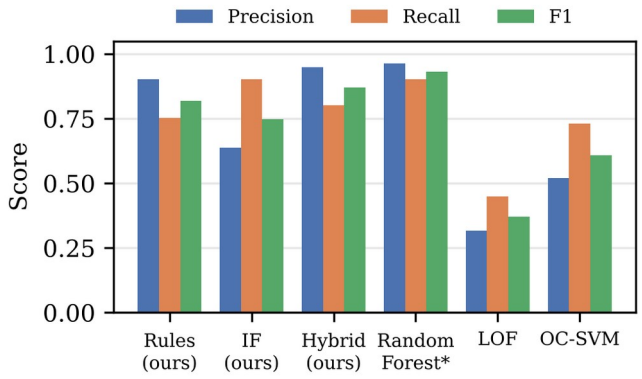


Fig. 4. Precision, recall and F1-score of each model in Experiment A. *Random Forest is the only model trained with labels.

Table III shows that fusion improves on both of its parts. The rule engine is precise but misses a quarter of the anomalous windows, and the Isolation Forest finds 90% of them at the cost of 192 false alarms. The hybrid keeps most of the rules' precision (0.950) while raising recall to 0.803, giving an F1-score of 0.870 and only 16 false alarms in 4,897 normal windows. Among methods that use no labels it is clearly the strongest: Local Outlier Factor and One-Class SVM reach F1-scores of 0.371 and 0.608 and raise 15 to 22 times more false alarms.

Random Forest achieves the best scores (F1 0.931, accuracy 0.991). This is expected: it is trained on 4,218 labelled windows that contain the same four incident types that appear in the test run. Accuracy is high for every model because 93% of windows are normal; F1-score and false alarms are more informative.

B. Detecting Incident Types Not Seen in Training

TABLE IV
Recall on an incident type absent from Random Forest training (Experiment B)

Unseen type	Random Forest	IF (ours)	Hybrid (ours)
Latency spike	1.000	1.000	1.000
Gradual slowdown	0.842	0.877	0.702
Error burst	0.000	0.849	0.822
Traffic drop	0.000	0.844	0.688
Mean	**0.461**	**0.893**	**0.803**

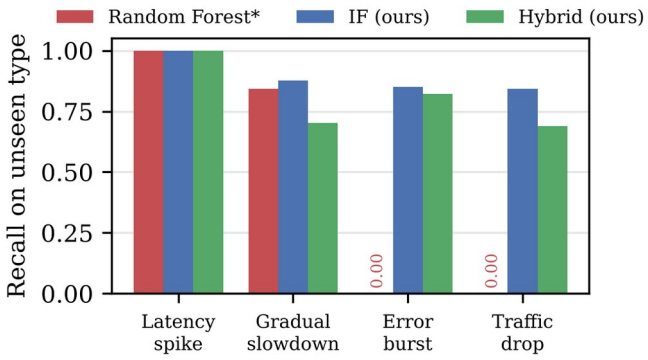


Fig. 5. Recall on an incident type that Random Forest never saw during training. Our detectors use no labels, so their results do not depend on which types occurred before.

Table IV changes the picture. When error bursts are missing from its training data, Random Forest detects none of them, and the same happens for traffic drops: it learned what the known incidents look like rather than what normal traffic looks like. Latency spikes and slowdowns are still caught because they resemble each other. Our Isolation Forest models normal behaviour only, so it recalls 84% to 100% of every type, 0.893 on average against 0.461 for Random Forest. The hybrid recalls 0.803 on average while keeping far fewer false alarms than the Isolation Forest alone.

C. Why the Hybrid Approach Is Preferable in Practice

The comparison supports the design for operational use. First, it needs no labels: Random Forest required 4,218 labelled windows per run, which a new deployment does not have. Second, it keeps detecting failure modes that have never been observed, where the supervised model's recall dropped to zero. Third, every alert is explained by the rules, values and thresholds that caused it, while a Random Forest vote does not tell an operator what is wrong. Fourth, it adapts by retraining on recent unlabelled data in under a second. When large labelled histories exist, a supervised model can be added as a further input to the fusion step.

D. Incident Detection and Investigation

In every run all four injected incidents produced alerts; the gradual slowdown was partly caught by model_only alerts that no rule raised. For the cascading failure, the investigator was run five times with different background traffic. In all five runs it ranked the user service as the root cause with 97-99% confidence and reconstructed the correct order of failure (user, then authentication, then payment service). Fig. 6 shows one report: the user service degraded first, both dependent services failed afterwards, and their error messages name it. An earlier version took a single noisy minute as the onset and blamed the authentication service; requiring two consecutive abnormal minutes removed this error.

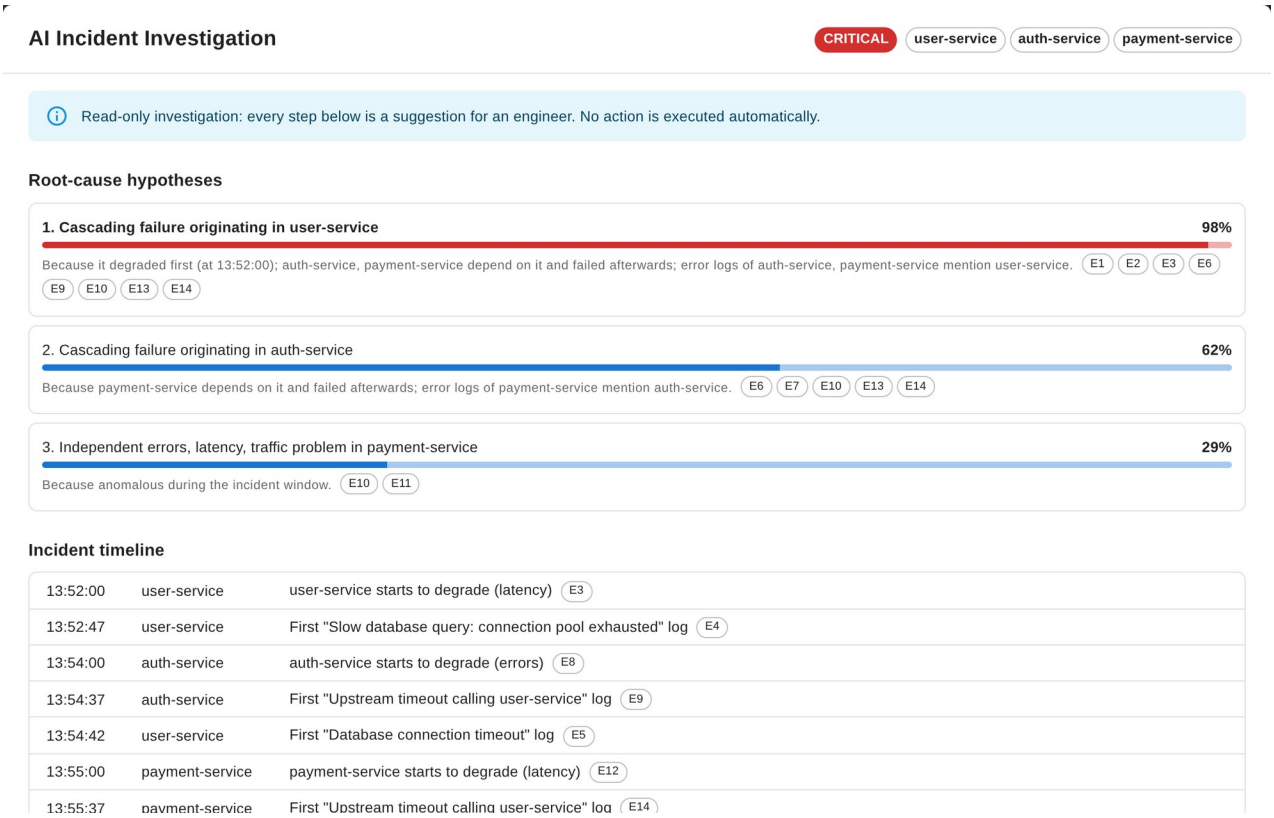


Fig. 6. Incident investigation of a simulated cascading failure: ranked root-cause hypotheses with evidence identifiers and the reconstructed timeline.

With the 3-billion-parameter llama3.2 model, reports followed the requested structure but sometimes omitted citations or suggested generic actions such as restarting services. The validation step listed the uncited sentences and

flagged the restart suggestion for human approval, which illustrates why the conclusion is computed from data and the language model is used only to explain it.

VIII. Limitations and Future Work

The evaluation uses generated traffic with injected incidents, so absolute scores on production systems may differ; the relative behaviour of supervised and unsupervised models on unseen incident types is the main finding. Window labels mark any window that overlaps an incident, which slightly lowers the measured recall at incident boundaries. The dependency map is configured by hand, the dashboard polls every 30 seconds, and the API has no authentication. Future work includes learning dependencies from traces, operator feedback on alerts, webhook notifications, and evaluation on public microservice benchmarks that include latency.

IX. Conclusion

We presented a MongoDB-based system that detects anomalies in microservice logs by fusing a severity-weighted rule engine with an unsupervised Isolation Forest and that investigates incidents with an evidence-grounded local language model. Without labels, the hybrid detector reached an F1-score of 0.870 with 16 false alarms in about 4,900 normal windows, well ahead of other unsupervised methods. A supervised Random Forest scored higher only when it had been trained on labelled examples of every incident type, and it missed all error bursts and traffic drops that were absent from its training data, while our Isolation Forest still detected 84-85% of them. The investigator located the origin of a cascading failure in every trial and never executes actions, which makes the approach suitable for environments where failure labels are rare and explanations are required.

References

[1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in Proc. IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413–422.

[2] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[3] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “LOF: Identifying density-based local outliers,” in Proc. ACM SIGMOD Int. Conf. Management of Data, 2000, pp. 93–104.

[4] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, “Estimating the support of a high-dimensional distribution,” Neural Computation, vol. 13, no. 7, pp. 1443–1471, 2001.

[5] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, “Detecting large-scale system problems by mining console logs,” in Proc. ACM Symp. Operating Systems Principles (SOSP), 2009, pp. 117–132.

[6] J.-G. Lou, Q. Fu, S. Yang, Y. Xu, and J. Li, “Mining invariants from console logs for system problem detection,” in Proc. USENIX Annual Technical Conf., 2010.

[7] S. He, J. Zhu, P. He, and M. R. Lyu, “Experience report: System log analysis for anomaly detection,” in Proc. IEEE Int. Symp. Software Reliability Engineering (ISSRE), 2016, pp. 207–218.

[8] M. Du, F. Li, G. Zheng, and V. Srikumar, “DeepLog: Anomaly detection and diagnosis from system logs through deep learning,” in Proc. ACM Conf. Computer and Communications Security (CCS), 2017, pp. 1285–1298.

[9] W. Meng et al., “LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs,” in Proc. Int. Joint Conf. Artificial Intelligence (IJCAI), 2019, pp. 4739–4745.

[10] H. Guo, S. Yuan, and X. Wu, “LogBERT: Log anomaly detection via BERT,” in Proc. Int. Joint Conf. Neural Networks (IJCNN), 2021.

[11] V.-H. Le and H. Zhang, “Log-based anomaly detection with deep learning: How far are we?” in Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE), 2022, pp. 1356–1367.

[12] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, “Recommending root-cause and mitigation steps for cloud incidents using large language models,” in Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE), 2023.

[13] Y. Chen et al., “Automatic root cause analysis via large language models for cloud incidents,” in Proc. European Conf. Computer Systems (EuroSys), 2024.

[14] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, “A survey on automated log analysis for reliability engineering,” ACM Computing Surveys, vol. 54, no. 6, 2021.

[15] K. Banker, P. Bakkum, S. Verch, D. Garrett, and T. Hawkins, MongoDB in Action, 2nd ed. Shelter Island, NY, USA: Manning, 2016.

[16] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[17] Z. Ji et al., “Survey of hallucination in natural language generation,” ACM Computing Surveys, vol. 55, no. 12, 2023.